It-Works-On-My-Machine

Jacob Choi & Owen Ungaro

01

The Problem

SQL's GROUP BY can only bucket rows one way at a time, grouping by customer is fine, but the moment you need aggregates across multiple subsets of that same group, standard SQL has no way to express it in one query.

01

One query per subset

Each state filter needs its own subquery with its own GROUP BY and a full table scan.

02

Results in separate sets

Getting NY, NJ, and CT side by side in one row per customer requires joining all three back together.

03

Doesn't scale

Adding a fourth dimension means another subquery and another join. N dimensions = N subqueries.

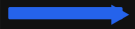
02

The Solution

Write a Phi operator spec. Run the engine. Get a standalone Python program that scans the sales table and computes your aggregates.

INPUT query.txt

```
SELECT:  cust, 1_sum_quant ...
n:  3
V:  cust
sigma:  1.state='NY'
        2.state='NJ'
        3.state='CT'
G:  none
```



OUTPUT generated_mf_query.py

```
Scan 0: seed MF table
Scan 1: filter NY → sum_quant
Scan 2: filter NJ → sum_quant
Scan 3: filter CT → sum_quant
Apply HAVING filter
Project SELECT columns
Return sorted rows
```

03

The Query Spec

```
SELECT ATTRIBUTE(S):
cust, 1_sum_quant,
2_sum_quant, 3_sum_quant

NUMBER OF GROUPING VARIABLES(n):
3

GROUPING ATTRIBUTES(V):
cust

F-VECT([F]):
1_sum_quant, 2_sum_quant, 3_sum_quant

SELECT CONDITION-VECT([sigma]):
1.state='NY'  2.state='NJ'  3.state='CT'

HAVING_CONDITION(G):
none
```

S — Columns to return: raw attributes or named aggregates

n — Number of grouping variables; one scan pass each

V — Grouping key: what rows get bucketed by

F — Aggregate function variables/references

sigma — Per-variable WHERE filter applied before aggregating

G — HAVING-style condition post-aggregation (or 'none')

04

Parsing & Validation

parser.py validates every field and produces a structured query_data dict. Bad input exits with a clear message (no silent failures).

V	Every attribute must exist in VALID_COLUMNS (the sales table schema)
F	Format is {group}_{agg}_{attr}: group must be in [1..n], agg must be sum/count/avg/min/max
S	Every aggregate in S must also be declared in F (no phantom references)
sigma	Exactly n entries, one per group; right-hand side can't reference a later group
G	AND-chain of comparisons; 'none' accepted to skip the HAVING filter entirely

05

The MF Table

mf_structure.py builds the in-memory structure that accumulates aggregate state across scans.

Entry Template

```
{
  'cust':          None,
  '1_sum_quant':   0,
  '2_sum_quant':   0,
  '3_sum_quant':   0,

  # avg: {sum, count, value}
  # max/min: None until set
}
```

- **01 One row per unique V**
  Scan 0 walks all sales rows and seeds the table with every distinct grouping-key combo.
- **02 Type-aware init**
  sum → 0, count → 0, avg → {sum,count,value}, max/min → None
- **03 Key-based lookup**
  lookup_mf_entry matches by grouping key; add_mf_entry appends on a miss.

06

Query Execution

mf_query.py runs n + 1 passes over the sales table.

SCAN 0

Seed

Walk every sales row. For each unique V combination encountered, append a fresh empty entry to the MF table.

SCAN 1..n

Aggregate

For group g: filter rows by sigma[g], then update agg fields assigned to g (sum, count, avg, max, or min).

POST

Filter & Project

Apply G (HAVING). Project S columns from surviving entries. Sort by V. Return the final result rows.

07

Point A to Point B

1

`query.txt`

Write spec
in plain text

2

`main.py`

File / CLI /
interactive

3

`parser.py`

Validate &
build query_data

4

`mf_query.py`

n+1 scan
passes

5

`generator.py`

Emit Python
programs

08

Contributions

Owen

Co-developed the full pipeline via pair programming, equal contributor across all modules and query logic.

Jacob

Co-developed the full pipeline via pair programming, equal contributor across all modules and query logic.

Claude (Anthropic)

Assisted with debugging sessions, repeated code for error checking functions, and regex.